

FOUNDATION_DB - PostgreSQL Database Foundation

Developer implementation guide: FOUNDATION_DB_DEV_IMPL_GUIDE.md.

1. Purpose

This foundation defines how SOPHIX modules use PostgreSQL: cluster layout, module database placement, connection pooling, replication, migrations, outbox/inbox storage, and recovery rules.

Use this document when implementing:

- A module database.
- A database migration.
- Business tables that emit Kafka events.
- Outbox and inbox storage.
- Database connection configuration.
- Read-replica routing.

2. Ownership

FOUNDATION_DB owns:

- PostgreSQL cluster topology.
- Module-to-cluster placement.
- PgBouncer usage.
- Replication model.
- Migration conventions.
- Database naming and schema conventions.
- Outbox/inbox database placement.
- Cluster-level failure behavior.

FOUNDATION_DB does not own:

- Module table schemas. Those are owned by each DD.
- Kafka event payloads. See FOUNDATION_KAFKA.md.
- Binary file storage. See FOUNDATION_FILE_STORAGE.md.
- Secrets. Use the platform secret manager.

3. Cluster Model

SOPHIX uses three PostgreSQL clusters. Each cluster has one primary and one replica.

Cluster	Purpose	Hosted databases	Replication	Criticality
A	Config and catalog	plm, audit, keycloak	Async	Low/medium
B	Customer capture and operations	onboarding, field_ops	Async	Medium
C	Revenue and workflow	billing, billing_cycle_close, billing_dunning, billing_wallet, subscription, fulfillment, camunda	Sync	Highest

Cluster C is the money and state cluster. It uses synchronous replication to avoid losing committed billing, subscription, fulfillment, or workflow state.

Infrastructure rule: keep the six PostgreSQL-node topology from day one. The database layer is critical enough that the project should not collapse clusters A, B, and C into fewer physical HA pairs for budget reasons.

4. Architecture

```
flowchart LR
    subgraph A["Cluster A - Config and Catalog"]
        PBA[PgBouncer A]
        PBA --> AP["Primary A"]
        AP -. async WAL .-> AR["Replica A"]
    end

    subgraph B["Cluster B - Capture and Ops"]
        PBB[PgBouncer B]
        PBB --> BP["Primary B"]
        BP -. async WAL .-> BR["Replica B"]
    end

    subgraph C["Cluster C - Revenue and Workflow"]
        PBC[PgBouncer C]
        PBC --> CP["Primary C"]
        CP == sync WAL ==> CR["Replica C"]
    end

    PLM[plm] --> PBA
    ONB[onboarding] --> PBB
    BILL[billing] --> PBC
    SUB[subscription] --> PBC
    FUL[fulfillment] --> PBC
    CAM[camunda] --> PBC
```

Applications must connect through PgBouncer. They must not connect directly to PostgreSQL.

5. Module Placement Rules

Module	Database	Cluster	Reason
PLM/catalog modules	plm	A	Low write rate, admin-heavy
Audit	audit	A	Append-only, replay tolerant
Keycloak	keycloak	A	Auth state, not revenue-critical after JWT issue
Onboarding	onboarding	B	Bursty capture workload
Field operations	field_ops	B	Bursty operational workload
Billing core and satellites	billing	C	Money-critical
BIL-03 Cycle Close	billing_cycle_close	C	Scanner-heavy, subscription/billing adjacent
BIL-04 Dunning	billing_dunning	C	Calls billing, wallet, subscription workflows
BIL-05 Wallet	billing_wallet	C	Money ledger, high write volume
Subscription	subscription	C	Central lifecycle state
Fulfillment	fulfillment	C	Activation and service execution state
Camunda	camunda	C	Long-lived workflow state

Cross-module data access rule:

```
No cross-database joins. Use REST APIs or Kafka events.
```

No module may create a foreign key to another module database.

6. PgBouncer

Every cluster has its own PgBouncer.

Common settings:

```
[pgbouncer]
listen_addr = 0.0.0.0
listen_port = 6432
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt
```

```

pool_mode = transaction
server_tls_sslmode = require
client_tls_sslmode = require

max_client_conn = 1000
default_pool_size = 25

server_idle_timeout = 600
server_lifetime = 3600
query_wait_timeout = 30

```

Example cluster C database mapping:

```

[databases]
billing                = host=pg-c-primary dbname=billing pool_size=60
billing_cycle_close    = host=pg-c-primary dbname=billing_cycle_close pool_size=30
billing_dunning        = host=pg-c-primary dbname=billing_dunning pool_size=30
billing_wallet         = host=pg-c-primary dbname=billing_wallet pool_size=60
subscription          = host=pg-c-primary dbname=subscription pool_size=40
fulfillment            = host=pg-c-primary dbname=fulfillment pool_size=30
camunda                = host=pg-c-primary dbname=camunda pool_size=40

```

Spring Boot Datasource

```

spring:
  datasource:
    url: jdbc:postgresql://pgbouncer-c.sophix.internal:6432/billing
    username: ${DB_USER}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 20

```

Laravel Datasource

```

'pgsql' => [
  'host' => env('DB_HOST', 'pgbouncer-b.sophix.internal'),
  'port' => env('DB_PORT', 6432),
  'database' => env('DB_DATABASE', 'onboarding'),
  'username' => env('DB_USERNAME'),
  'password' => env('DB_PASSWORD'),
  'sslmode' => 'require',
],

```

7. Replication

All clusters use PostgreSQL streaming replication via WAL.

Common primary settings:

```

wal_level = replica
max_wal_senders = 5
max_replication_slots = 5
wal_keep_size = 1024
archive_mode = on
archive_command = 'rsync %p /var/lib/postgresql/wal_archive/%f'
hot_standby = on
hot_standby_feedback = on

```

Cluster A and B:

```

synchronous_commit = local
synchronous_standby_names = ''

```

Cluster C:

```

synchronous_commit = on
synchronous_standby_names = 'FIRST 1 (pg-c-replica)'

```

Cluster C write rule: writes return only after primary and replica persist WAL. This gives zero RPO for committed revenue and lifecycle transactions.

8. Migrations

Stack	Migration tool	Location
Java/Spring	Liquibase	src/main/resources/config/liquibase/changeLog/
Laravel	Eloquent migrations	database/migrations/

Migration rules:

Rule	Description
DB-MIG-1	Migrations must be versioned and committed with the module code.
DB-MIG-2	Prefer additive migrations.
DB-MIG-3	Do not drop a column/table without a deprecation migration and rollout plan.
DB-MIG-4	Backfills for large tables must be batched.
DB-MIG-5	Index creation on large tables must use non-blocking mechanisms where supported.
DB-MIG-6	Every table must have audit columns unless explicitly justified.

9. Schema Conventions

Item	Convention
Table names	snake_case
Primary keys	ULID or UUID v7 stored as TEXT
Timestamp columns	TIMESTAMPTZ
Audit columns	created_at, created_by, updated_at, updated_by
Soft delete	retired_at or deleted_at nullable timestamp
Flexible attributes	JSONB
Cross-module references	String IDs only, resolved by API
Cross-module FK	Not allowed

Recommended base columns:

```
id          TEXT PRIMARY KEY,  
created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
created_by  TEXT NOT NULL,  
updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
updated_by  TEXT NOT NULL,  
retired_at  TIMESTAMPTZ NULL
```

10. Outbox and Inbox Storage

Each module stores outbox and inbox tables in its own database, in schema messaging.

```
CREATE SCHEMA IF NOT EXISTS messaging;
```

Producer outbox:

```
CREATE TABLE messaging.outbox_event (  
  event_id      TEXT PRIMARY KEY,  
  topic         TEXT NOT NULL,  
  message_key   TEXT NOT NULL,  
  envelope      JSONB NOT NULL,  
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  published_at  TIMESTAMPTZ NULL,  
  attempts      INTEGER NOT NULL DEFAULT 0,  
  last_error    TEXT NULL  
);  
  
CREATE INDEX idx_outbox_pending  
  ON messaging.outbox_event (created_at)  
  WHERE published_at IS NULL;
```

Consumer inbox:

```
CREATE TABLE messaging.inbox_event (  
  event_id      TEXT PRIMARY KEY,  
  topic         TEXT NOT NULL,  
  message_key   TEXT NOT NULL,  
  envelope      JSONB NOT NULL,  
  received_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  processed_at  TIMESTAMPTZ NULL,  
  attempts      INTEGER NOT NULL DEFAULT 0,  
  last_error    TEXT NULL  
);  
  
CREATE INDEX idx_inbox_pending
```

```
ON messaging.inbox_event (received_at)
WHERE processed_at IS NULL;
```

Retention:

Table	Retention
messaging.outbox_event	30 days after published_at
messaging.inbox_event	7 days after processed_at

High-volume modules should partition outbox/inbox by month.

11. Transaction Pattern

When a business write emits an event, write both records in the same database transaction.

```
BEGIN;

INSERT INTO invoice (...);

INSERT INTO messaging.outbox_event (
  event_id,
  topic,
  message_key,
  envelope
) VALUES (
  'billing:invoice:inv_01HX:evt_01HY',
  'sophix.billing.invoice.issued',
  'billing:invoice:inv_01HX',
  '{...}'::jsonb
);

COMMIT;
```

If the transaction commits, the event is durable even if Kafka is unavailable.

12. Read Replica Usage

Use replica reads only for queries that tolerate staleness.

Good candidates:

- Reporting pages.
- Admin dashboards.
- Reconciliation previews.
- Historical invoice lists.

Do not use replicas for:

- Payment posting.
- Wallet debit/credit.
- Subscription status commit.
- Workflow state transition.
- Idempotency checks.

Example read-only datasource:

```
spring:
  datasource-replica:
    url: jdbc:postgresql://pgbouncer-c-replica.sophix.internal:6432/billing_ro
```

13. Failure Behavior

Failure	Behavior
Cluster A down	Catalog admin and auth login degrade. Runtime may continue from Redis caches and existing JWTs.
Cluster B down	New capture and field operations stop. Existing revenue flows continue.

Failure	Behavior
Cluster C down	Payments, invoicing, activation, subscription workflows, fulfillment, and Camunda stop.
Replica down on A/B	Primary continues. Restore replica from base backup and WAL.
Replica down on C	Writes may block. Ops may temporarily disable sync mode under incident procedure.
Primary down	Replica promotes; PgBouncer points to new primary; apps reconnect.
Data corruption	Restore PITR clone, extract corrected rows, patch live primary through audited procedure.

14. Monitoring

Monitor each cluster for:

Signal	Alert threshold
Replication lag, async clusters	More than 5 seconds
Replication lag, sync cluster	More than 1 second
PgBouncer pool usage	More than 80 percent
Disk usage	More than 75 percent warning, 90 percent critical
WAL archive failure	Any failure
Primary unreachable	Any failure
Backup verification	Any failure
Slow query rate	Sustained growth
Pending outbox rows older than 1 minute	Any sustained count
Pending inbox rows older than 5 minutes	Any sustained count

15. Medium Budget Infrastructure

This section defines the recommended starting infrastructure for a medium deployment with limited budget. The sizing is conservative, but the topology remains six PostgreSQL nodes because the database layer is critical.

15.1 Target vs Budget Phase

Required production architecture:

```
Cluster A: primary + replica
Cluster B: primary + replica
Cluster C: primary + replica
```

Medium budget means smaller node dimensions, not fewer database nodes.

Do not run one large shared PostgreSQL cluster for all SOPHIX databases in production. That saves servers but creates one blast radius for catalog, capture, billing, subscription, fulfillment, and workflow.

15.2 Recommended Server Dimensions

Medium budget baseline:

Server role	Count	CPU	RAM	Disk	Notes
Cluster A PostgreSQL primary/replica	2	4 vCPU each	16 GB each	500 GB SSD each	Hosts config/catalog/auth/audit. Async replication.
Cluster B PostgreSQL primary/replica	2	4 vCPU each	16 GB each	750 GB SSD each	Hosts customer capture and field operations. Async replication.
Cluster C PostgreSQL primary/replica	2	8 vCPU each	32 GB each	1 TB SSD/NVMe each	Hosts revenue, subscription, fulfillment, Camunda. Sync replication.
PgBouncer	2 preferred	1-2 vCPU each	2 GB each	20 GB SSD each	Can run as dedicated small VMs or co-located if budget forces it.

Server role	Count	CPU	RAM	Disk	Notes
Backup/monitoring storage	Shared platform service	2-4 vCPU	8 GB	Depends on retention	Must be outside the DB nodes for real backup value.

Minimum acceptable production start:

2 x Cluster A DB nodes: 4 vCPU / 16 GB RAM / 500 GB SSD
 2 x Cluster B DB nodes: 4 vCPU / 16 GB RAM / 750 GB SSD
 2 x Cluster C DB nodes: 8 vCPU / 32 GB RAM / 1 TB SSD or NVMe
 1 x PgBouncer/LB node: 2 vCPU / 2 GB RAM / 20 GB SSD

Preferred medium start:

2 x Cluster A DB nodes: 4 vCPU / 16 GB RAM / 500 GB SSD
 2 x Cluster B DB nodes: 4-6 vCPU / 16-24 GB RAM / 1 TB SSD
 2 x Cluster C DB nodes: 8-12 vCPU / 32-48 GB RAM / 1-2 TB NVMe
 2 x PgBouncer/LB nodes: 2 vCPU / 2 GB RAM / 20 GB SSD

Placement rules:

- Do not place a primary and its replica on the same physical host.
- Do not place all primaries on the same physical host.
- If using virtualization, anti-affinity must keep each pair split across hosts.
- Place Cluster C on the best disks available.
- PgBouncer may be co-located in the first phase, but dedicated PgBouncer nodes are preferred once traffic grows.
- Backups must leave the DB node. A local backup directory is only staging, not final backup storage.

15.3 Recommended OS

Use one operator-standard Linux distribution:

OS	Recommendation
Ubuntu Server 22.04 LTS or 24.04 LTS	Recommended default for small/medium teams. Good package availability and operational familiarity.
Rocky Linux 9 / RHEL 9	Use if the operator standardizes on RHEL-family systems.

OS baseline:

- 64-bit minimal server install.
- No desktop packages.
- Security patching through unattended upgrades or the operator patch window.
- chrony or equivalent NTP enabled.
- SSH restricted to admin VPN/network.
- Host firewall enabled.
- PostgreSQL version standardized across all DB nodes in the deployment.
- Same OS and PostgreSQL minor version on primary and replica.

15.4 Disk Partitioning

Use SSD for Cluster AB and SSD/NVMe for Cluster C.

Filesystem:

- xfs or ext4.
- Use LVM if the ops team is comfortable with it.
- Avoid complex storage layouts if the local team will struggle to operate them.

Mount recommendations:

Mount	Purpose
/	OS and packages

Mount	Purpose
/var/lib/postgresql	PostgreSQL data directory
/var/lib/postgresql/wal_archive	Local WAL archive staging
/var/log	PostgreSQL, PgBouncer, and OS logs
/backup	Optional local backup staging only

Cluster A with 500 GB disk:

```

/                               40 GB
/var/lib/postgresql             320 GB
/var/lib/postgresql/wal_archive  80 GB
/var/log                        20 GB
/backup                         40 GB
free/reserved                   remaining

```

Cluster B with 750 GB disk:

```

/                               40 GB
/var/lib/postgresql             520 GB
/var/lib/postgresql/wal_archive 120 GB
/var/log                        25 GB
/backup                         45 GB
free/reserved                   remaining

```

Cluster C with 1 TB disk:

```

/                               50 GB
/var/lib/postgresql             700 GB
/var/lib/postgresql/wal_archive 150 GB
/var/log                        30 GB
/backup                         70 GB
free/reserved                   remaining

```

Cluster C with 2 TB disk:

```

/                               50 GB
/var/lib/postgresql             1450 GB
/var/lib/postgresql/wal_archive 250 GB
/var/log                        50 GB
/backup                         150 GB
free/reserved                   remaining

```

Partition rules:

- Do not let PostgreSQL data, WAL archive, and logs share a tiny root filesystem.
- Alert at 75 percent disk usage and treat 90 percent as critical.
- WAL archive must be monitored. A full WAL archive filesystem can stop PostgreSQL.
- Local /backup is optional and only for staging. Final backups must be copied to separate backup storage.

15.5 PostgreSQL Runtime Recommendations

Initial settings must be tuned after load testing, but the following starting points are reasonable for the medium profile.

Cluster A:

```

shared_buffers = 4GB
effective_cache_size = 10GB
maintenance_work_mem = 512MB
work_mem = 16MB
max_connections = 200
wal_level = replica
max_wal_senders = 5
max_replication_slots = 5

```

Cluster B:

```

shared_buffers = 4GB
effective_cache_size = 10GB
maintenance_work_mem = 512MB
work_mem = 16MB
max_connections = 200
wal_level = replica
max_wal_senders = 5
max_replication_slots = 5

```

Cluster C:


```

shared_buffers = 8GB
effective_cache_size = 24GB
maintenance_work_mem = 1GB
work_mem = 16MB
max_connections = 300
wal_level = replica
max_wal_senders = 5
max_replication_slots = 5
synchronous_commit = on
synchronous_standby_names = 'FIRST 1 (pg-c-replica)'

```

Connection rule: keep application-side pools small and let PgBouncer absorb concurrency. Too many direct PostgreSQL connections will hurt performance.

15.6 Network Ports

Flow	Port	Exposure
App to PgBouncer	6432	Internal application network
PgBouncer to PostgreSQL	5432	DB network only
Primary to replica streaming replication	5432	DB network only
Admin SSH	22	Admin VPN/network only
Monitoring scrape	Operator standard	Monitoring network only
Backup copy to storage	Operator standard	Backup network only

16. AWS Deployment Recommendations

This section applies only when deploying the database foundation on AWS. The logical requirement remains the same: three separated PostgreSQL clusters, each with high availability.

16.1 Recommended AWS Shape

Preferred managed option:

SOPHIX cluster	AWS service	Deployment
Cluster A	Amazon RDS for PostgreSQL	Multi-AZ
Cluster B	Amazon RDS for PostgreSQL	Multi-AZ
Cluster C	Amazon RDS for PostgreSQL	Multi-AZ, highest class/storage priority

Use three separate RDS instances or Multi-AZ DB clusters, not one shared database instance with all databases. Separation keeps the same blast-radius boundaries as the SOPHIX architecture.

Self-managed option:

Component	AWS service	Recommendation
PostgreSQL nodes	EC2	6 EC2 instances across at least 2 Availability Zones
Storage	EBS gp3 or io2	gp3 for A/B; gp3 high IOPS or io2 for C if budget allows
Failover	Patroni, pg_auto_failover, or operator standard	Required if not using RDS
Backups	EBS snapshots + PostgreSQL base backup/WAL archive to S3	Required

Use RDS unless there is a strong reason to self-manage PostgreSQL. RDS reduces operational burden for backups, patching, failover, and monitoring.

16.2 RDS Sizing

Medium budget start:

```

Cluster A RDS:
Class:    db.m7g.large or db.t4g.large
Storage:  500 GB gp3
HA:       Multi-AZ

Cluster B RDS:
Class:    db.m7g.large
Storage:  750 GB gp3

```

HA: Multi-AZ

Cluster C RDS:
Class: db.m7g.2xlarge or db.r7g.xlarge
Storage: 1 TB gp3, provisioned IOPS if load tests require it
HA: Multi-AZ

Preferred medium:

Cluster A RDS:
Class: db.m7g.large
Storage: 500 GB gp3

Cluster B RDS:
Class: db.m7g.xlarge
Storage: 1 TB gp3

Cluster C RDS:
Class: db.r7g.2xlarge
Storage: 1-2 TB gp3 with tuned IOPS, or io2 if write latency requires it

Use Graviton classes where supported by the chosen PostgreSQL version and region. If local team tooling is not ARM-ready, use Intel/AMD equivalents.

16.3 RDS Multi-AZ Choice

AWS has multiple Multi-AZ deployment shapes. Choose deliberately:

Option	Use when
Multi-AZ DB instance deployment	Good default for budget-conscious production HA. Standby is for failover, not normal reads.
Multi-AZ DB cluster deployment	Use when lower write latency and readable standbys are important, and the region/engine version supports it. Usually costs more.

For SOPHIX medium start:

- Cluster A: Multi-AZ DB instance is enough.
- Cluster B: Multi-AZ DB instance is enough.
- Cluster C: evaluate Multi-AZ DB cluster if billing/cycle-close load requires readable standbys or lower write latency.

16.4 PgBouncer on AWS

RDS does not replace PgBouncer for application connection pooling.

Recommended:

2 x small EC2 instances for PgBouncer
Instance: t4g.small/t3.small minimum, t4g.medium preferred
Placement: private app subnets across 2 AZs
Front: internal Network Load Balancer or Route53 failover

Alternative:

- Use RDS Proxy for some application pools if it fits the stack.
- Still document and test transaction-pooling behavior before replacing PgBouncer.

16.5 AWS Network Layout

Recommended VPC layout:

Private app subnets:
- Application services
- PgBouncer EC2 instances

Private DB subnets:
- RDS PostgreSQL

Public subnets:
- Public ALBs only, if needed by user-facing apps

Security group rules:

Source	Destination	Port	Purpose
App service SG	PgBouncer SG	6432	Application DB access

Source	Destination	Port	Purpose
PgBouncer SG	RDS SG	5432	PostgreSQL access
Admin VPN/bastion/SSM	PgBouncer EC2	22 or SSM	Admin access
Monitoring SG	PgBouncer/EC2 exporters	Operator standard	Metrics

Do not expose RDS publicly.

16.6 Backups and DR on AWS

Minimum:

- Enable automated RDS backups.
- Retain backups for at least 7 days; 30 days preferred for Cluster C.
- Enable deletion protection for production RDS instances.
- Take manual snapshots before major migrations.
- Copy critical snapshots to another AWS Region if disaster recovery requires it.
- Store PostgreSQL logical dumps for selected critical catalogs if the ops process requires human-readable restore artifacts.

For self-managed EC2 PostgreSQL:

- Archive WAL to S3.
- Take daily base backups.
- Test point-in-time restore.
- Do not rely only on EBS snapshots without PostgreSQL-aware backup discipline.

16.7 AWS Monitoring

Alarm on:

Signal	Threshold
RDS CPU	Sustained above 75 percent
RDS free storage	Below 20 percent
RDS freeable memory	Sustained low memory
RDS database connections	Above planned pool ceiling
RDS replica lag / replication health	Any sustained lag or degraded status
RDS write latency, Cluster C	Sustained increase above baseline
PgBouncer pool usage	Above 80 percent
Failed backups	Any

Reference docs:

- [Amazon RDS Multi-AZ deployments](https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html)
- [RDS Multi-AZ DB clusters](https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/multi-az-db-clusters-concepts.html)
- [Amazon EC2 instance types](https://docs.aws.amazon.com/ec2/latest/instancetypes/gp.html)

17. Developer Checklist

Before merging a database change:

- [] Module uses the correct database and cluster.
- [] App connects through PgBouncer, not directly to PostgreSQL.
- [] No cross-module foreign keys.
- [] Cross-module references are string IDs.

- [] Tables use snake_case.
- [] Primary keys use ULID or UUID v7.
- [] Audit columns exist.
- [] Migration is additive or has a deprecation plan.
- [] Business event writes use `messaging.outbox_event`.
- [] Consumers use `messaging.inbox_event`.
- [] High-volume tables have partitioning/index strategy.
- [] Tests cover idempotency and transaction rollback where events are emitted.